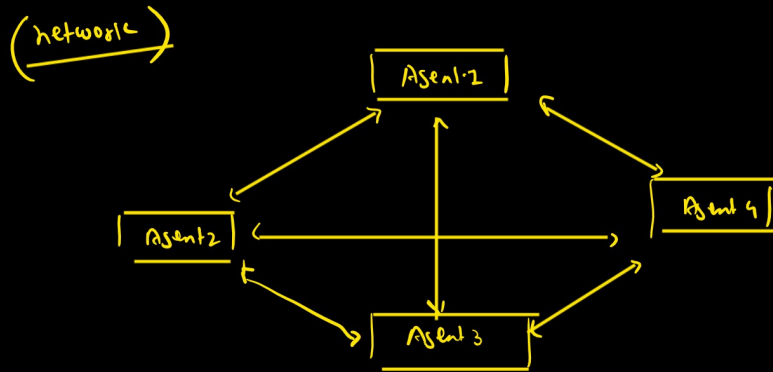
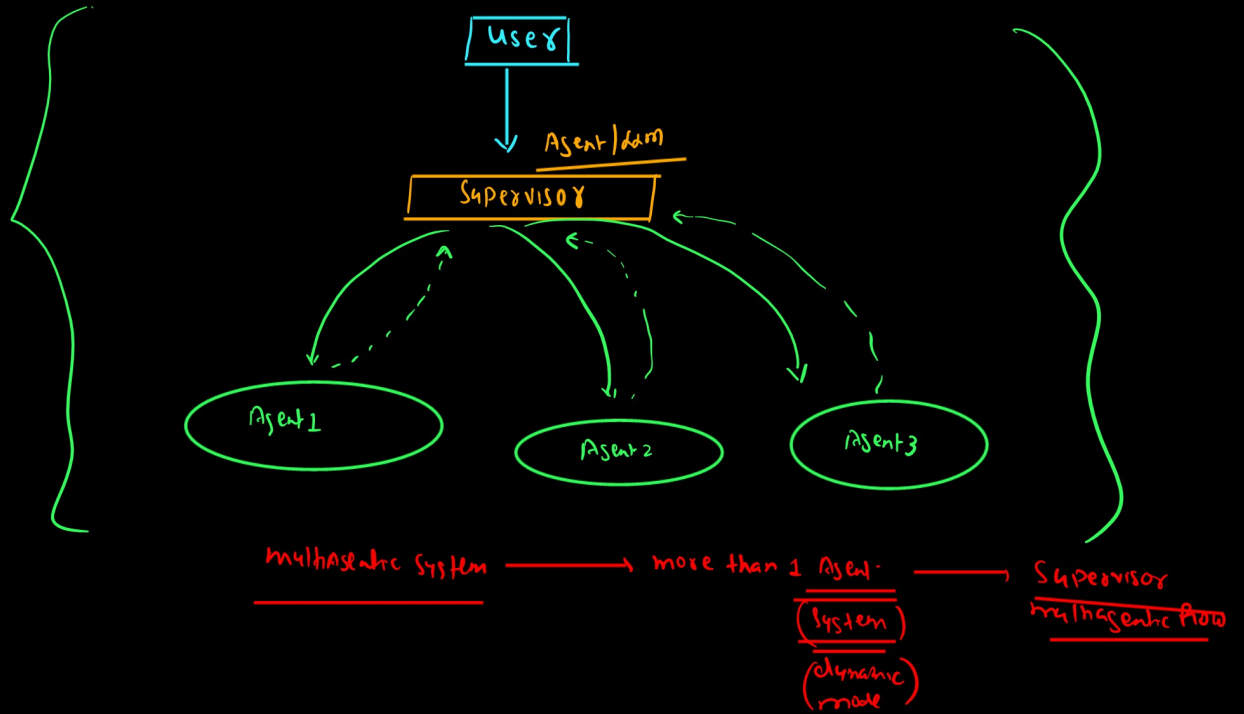


Diagram illustrating three types of agents (Agent 1, Agent 2, Agent 3) and their associated tools.

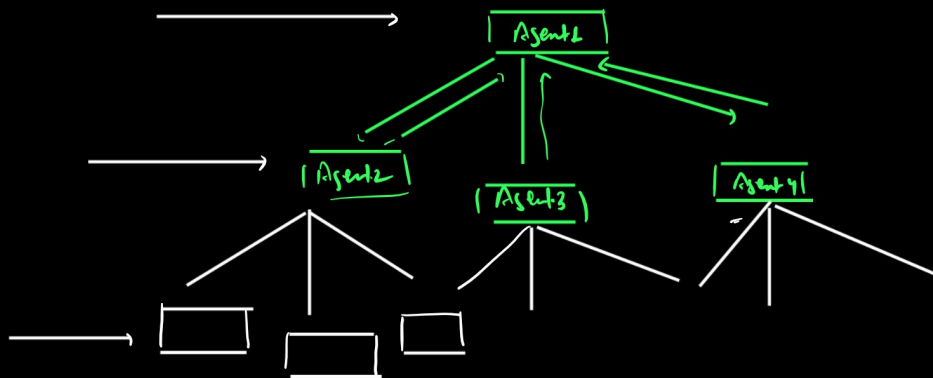
- Agent 1:** A central node labeled "dam" is connected to four peripheral nodes labeled "tool1", "tool2", "tool3", and "tool4".
- Agent 2:** A central node labeled "dam" is connected to four peripheral nodes labeled "tool1", "tool2", "tool3", and "tool4".
- Agent 3:** A central node labeled "dam" is connected to four peripheral nodes labeled "tool1", "tool2", "tool3", and "tool4".

- 1 network ✓
- 2 supervisor ✓
- 3 hierarchical ✓

{ Any Single Agent ———→ Deep Agent-
 Any multi Agent ———→



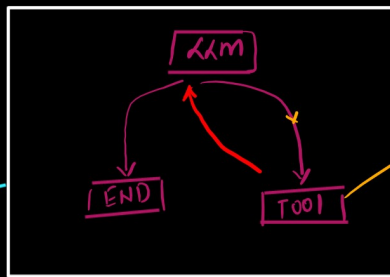
Hierarchical flow



Autonomy
LLM with tool calling
Dynamic basis

Re. Act

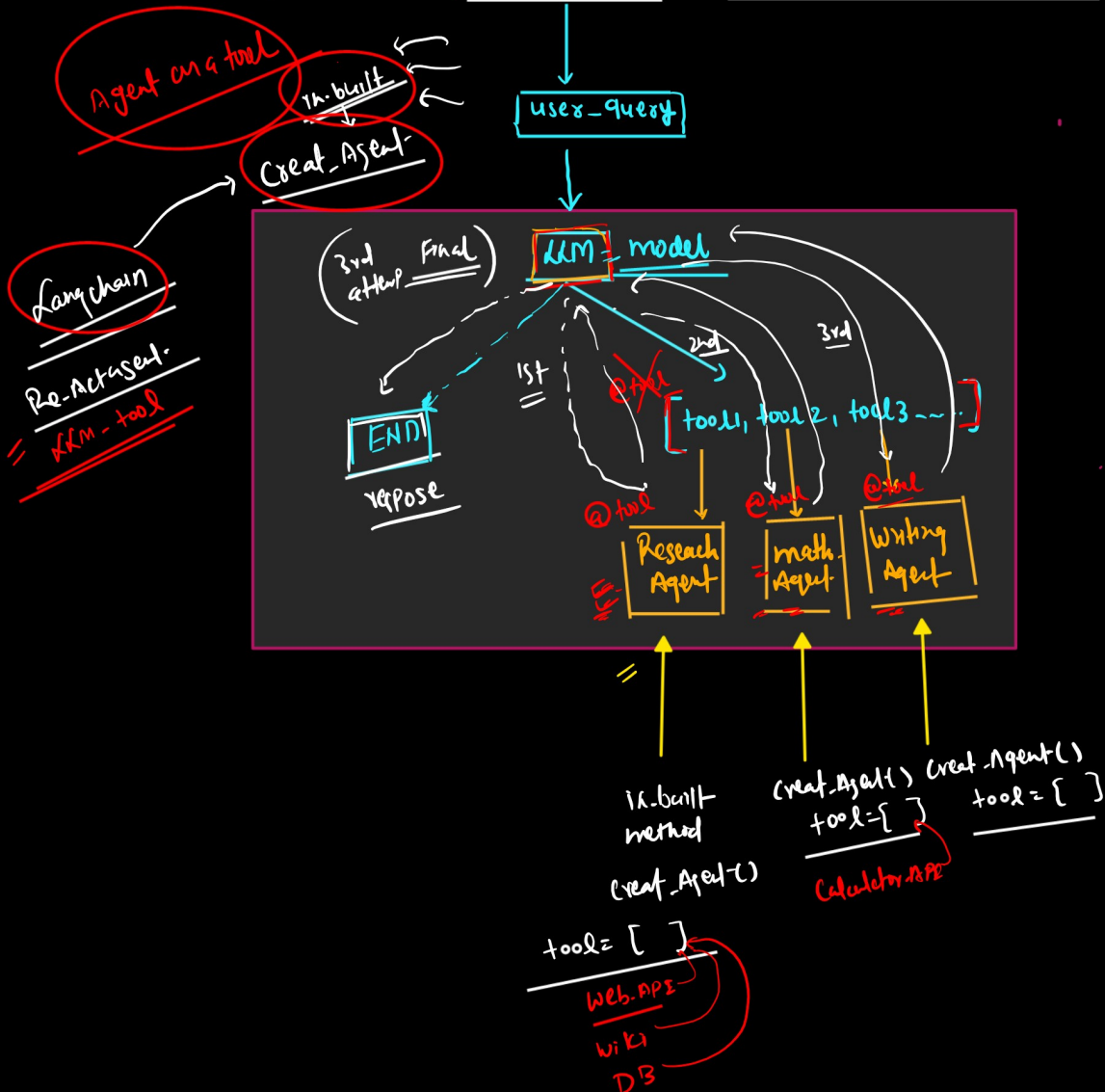
Reasoning - Action



(multiple tool)
Which tool needs to be call

from langchain.agents import create_agent

"Explain RAG briefly, calculate 25 * 16, and present the answer in a clean professional format."



```

main_delegation_agent = create_agent(
    model=model, LLM
    tools=[
        ask_research_agent, Read - tool-acan [ ]
        ask_math_agent, Read - tool-acan [ ]
        ask_writing_agent, Read - tool-acan [ ]
    ],
    system_prompt="""
You are the main coordinator.

```

You have three specialist subagents:

1. Research specialist
2. Math specialist
3. Writing specialist

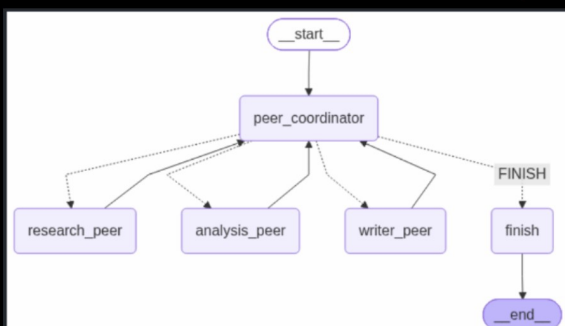
Delegate work to the appropriate specialist tools.
 You may call multiple specialists when necessary.
 Finally provide one clear answer to the user.

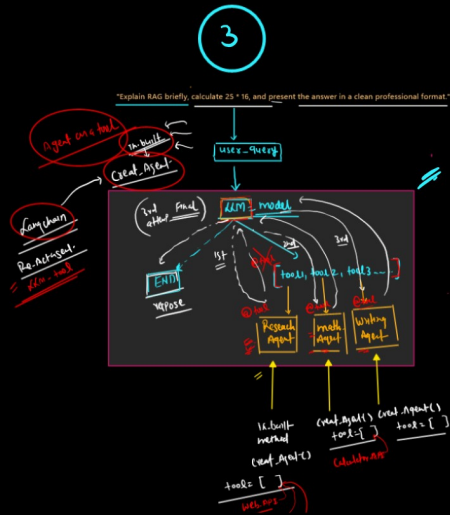
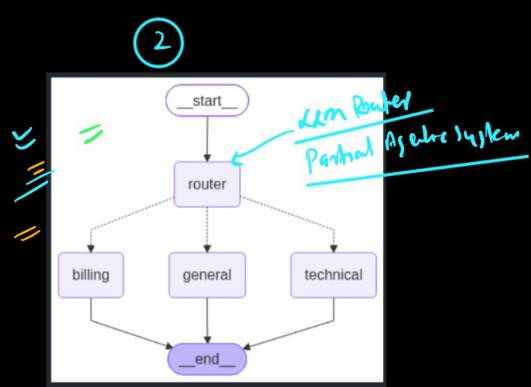
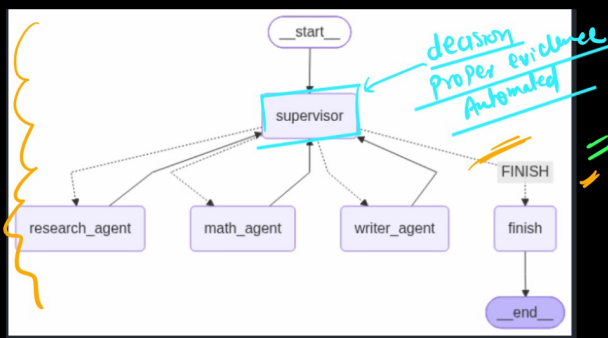
```

""",
    name="main_coordinator",
)

```

supervisor_router





Deep Apertic System (more - more reasoning)
(long horizon)

create-agent()

deep-set (?)

LangGraph → Command() → multiageticsystem

Telegraph
Internet
Send
Commenced

2-3 weeks §-6

1-m.
MCP/A2A → 2

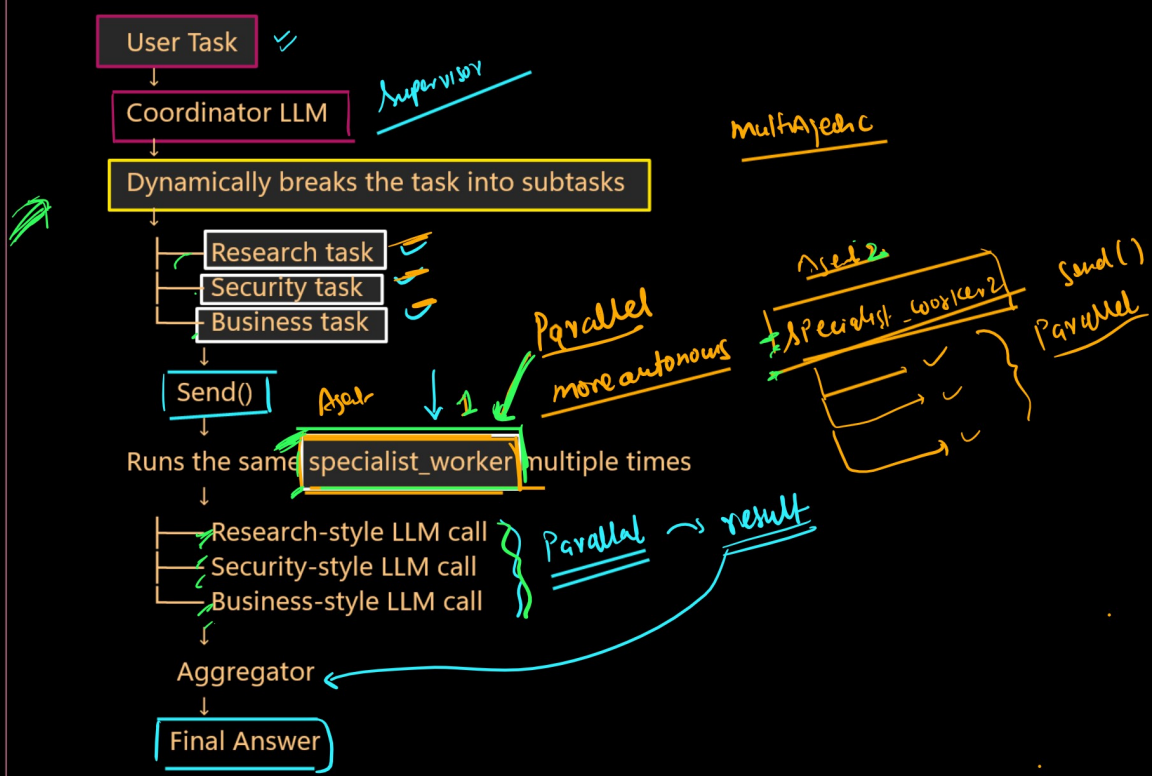
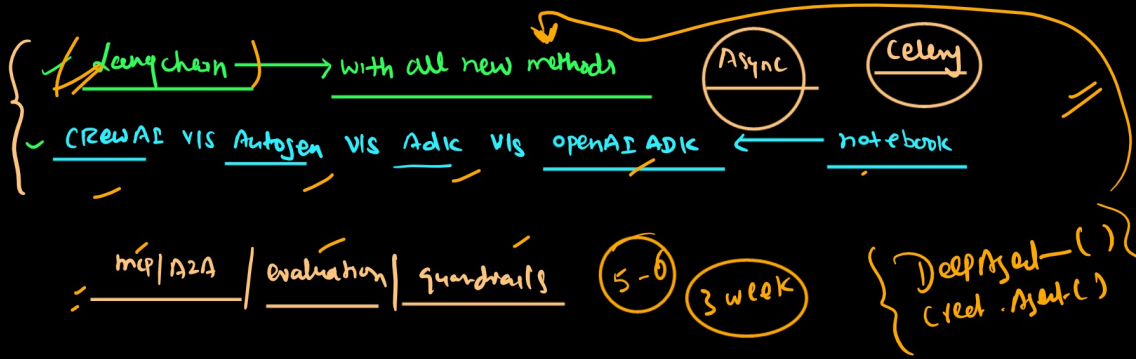
Blockchain

Diff - Agentische

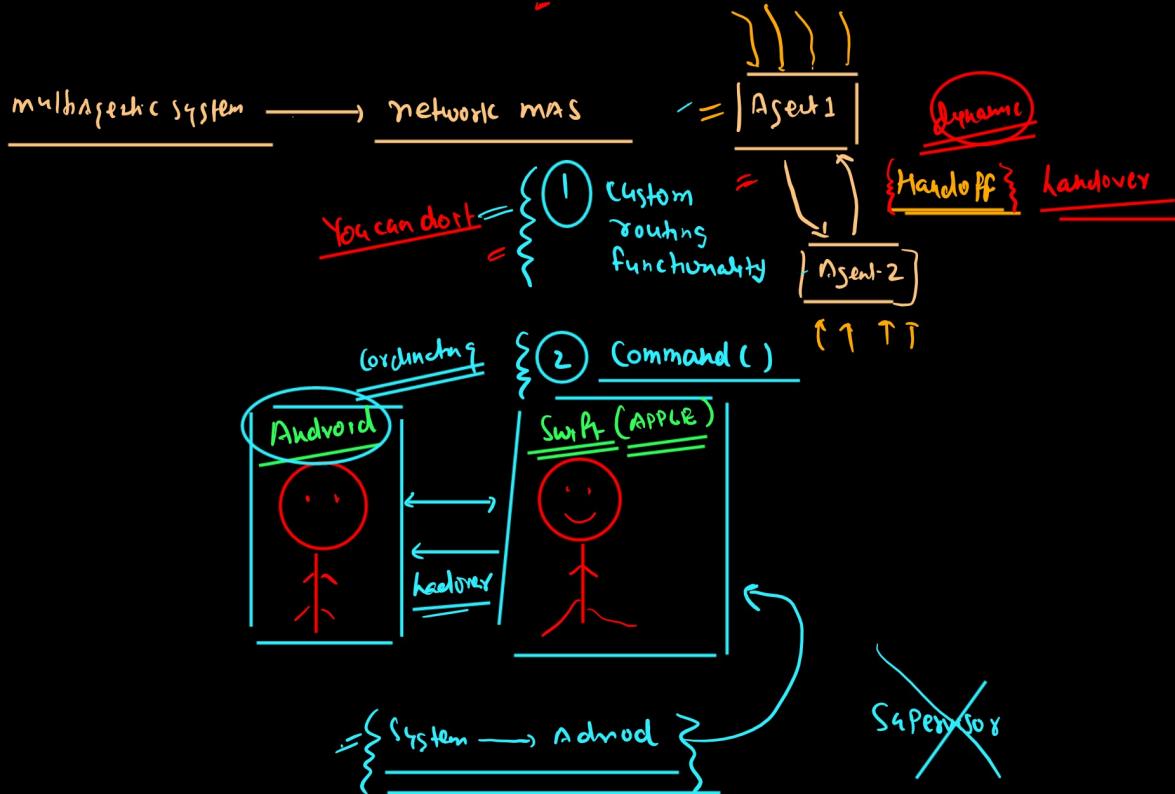
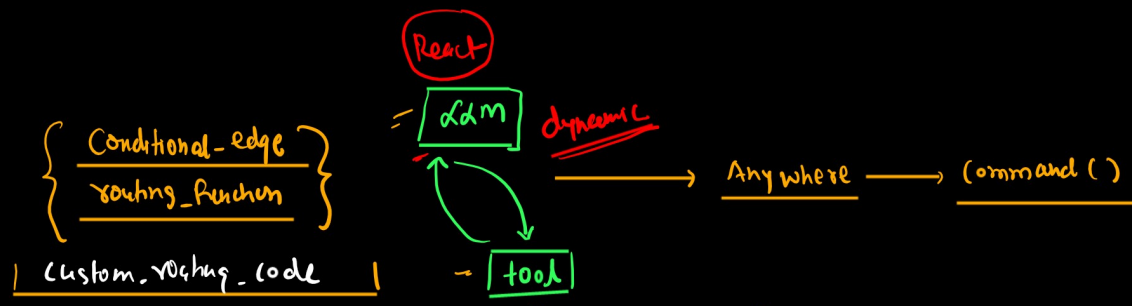
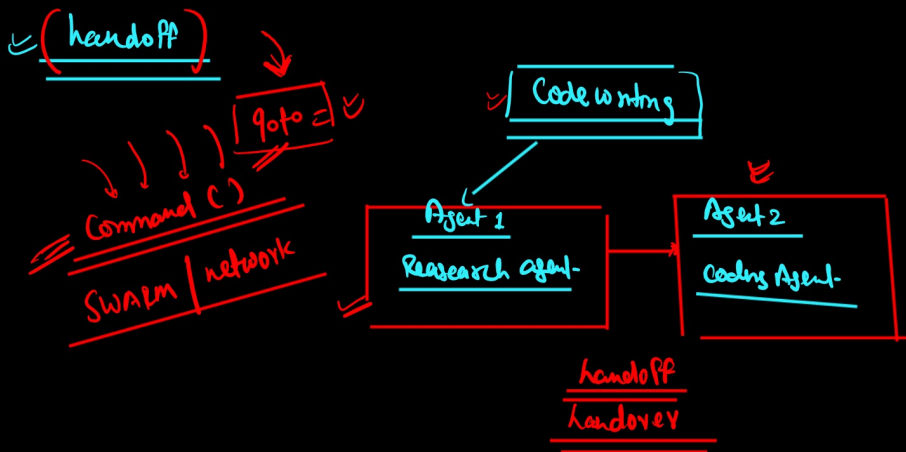
Interview

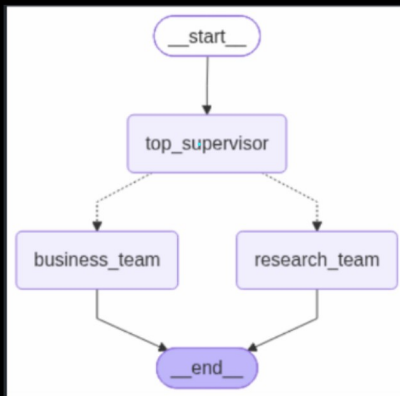
ADDER

ORANGE



Command() → not a mandatory fn
but let's understand the use of it





Hierarchical FLOW ✓

Supervisor Flow ✓

Complete Agent's Flow χ

Complete deterministic Flow X

hybrid flow (somewhat deterministic & dynamic case)

RAM enable workflow

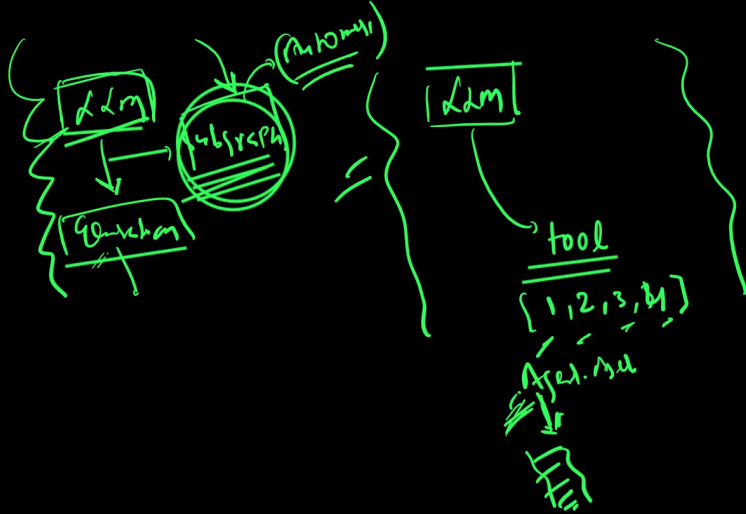
Supervisor

Subset of hierarchical flow

- LLM enable workflow
- more reliable & deterministic
- Partial Agentic flow

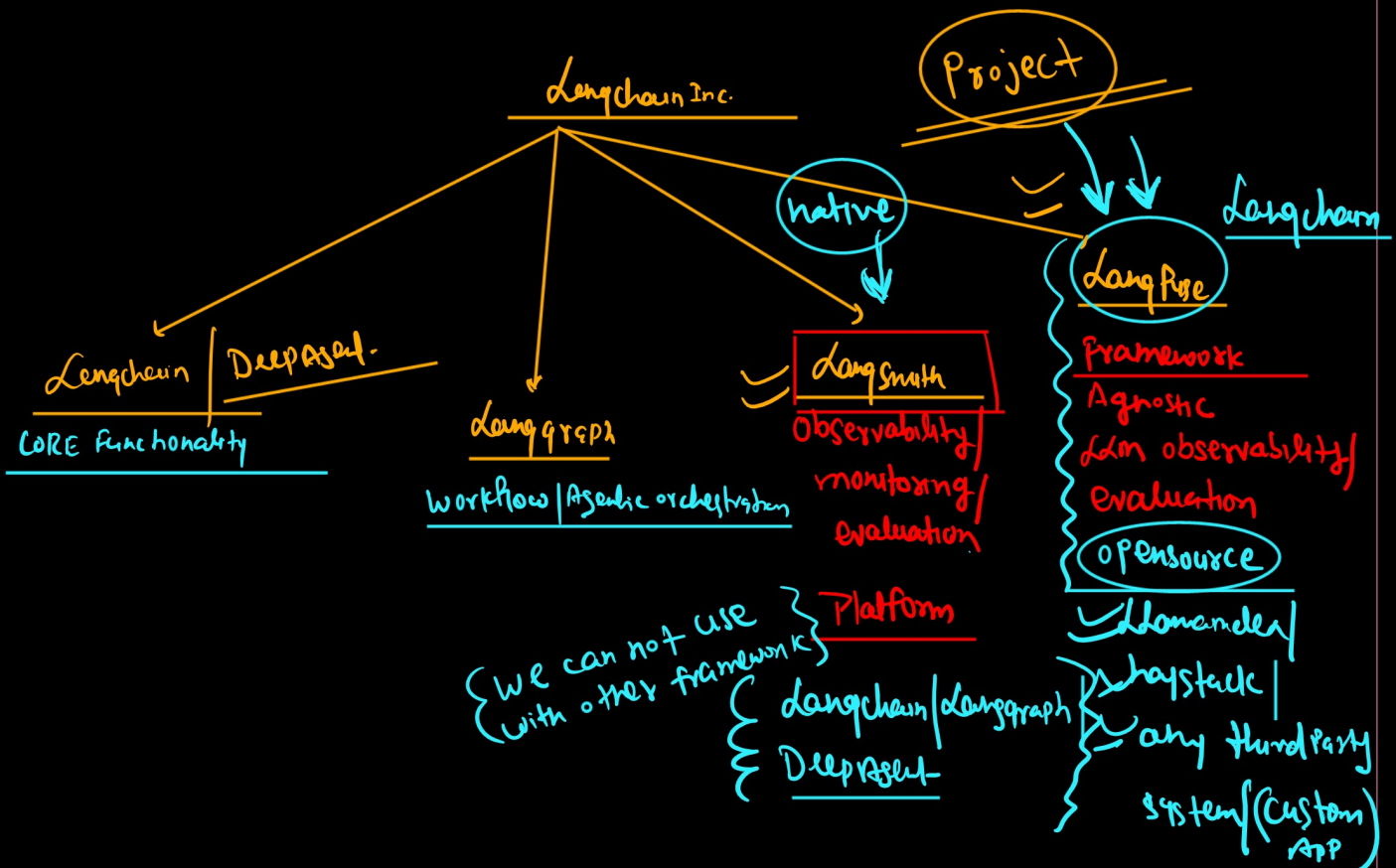
Enterprise / User Facing (complete autonomy)





- Documents ← Langchain document
 - chat models + model factory + profiles + streaming/batching ← model from diff Provider
streaming / batching
 - messages + multimodal/content blocks
 - prompts + placeholders + partials
 - output parsers + native structured output structured / json
 - Runnable interface + LCEL
 - retries/fallbacks/config/caching/rate limiting
 - document loaders = RAG
 - text splitters
 - embeddings
 - vector stores
 - retrievers + reranking patterns
 - tools + manual tool calling + ToolRuntime ← Async
inbuilt method
 - create_agent + structured agent output ← latest Langchain
 - middleware + context engineering
 - short/long-term memory
 - streaming + events memory manager
- { core langchain }
 { functionality }
 { base langchain }
 { functionality }
GenAI / Agentic AI

- MCP
- LangGraph (multi-agent patterns) separate module
- Deep Agents separate module
- LangSmith testing/evals/observability separate module
- Langfuse - opensource platform - Agentic/genai application observability + evaluation platform
← separate platform / module



Placeholder → Fill out

< ↑ >
≡ ≡

message placeholder → place the message at any pos

message history | memory

message history

Latest question

message Placeholder

Langgraph

Agentic AI

tool calling

LLM + tool

(Autonomous Agent System)

LLM dynamically decide which tool needs to be call.

Any functionality

this is most recommended

Agent

React

LLM

tool

N

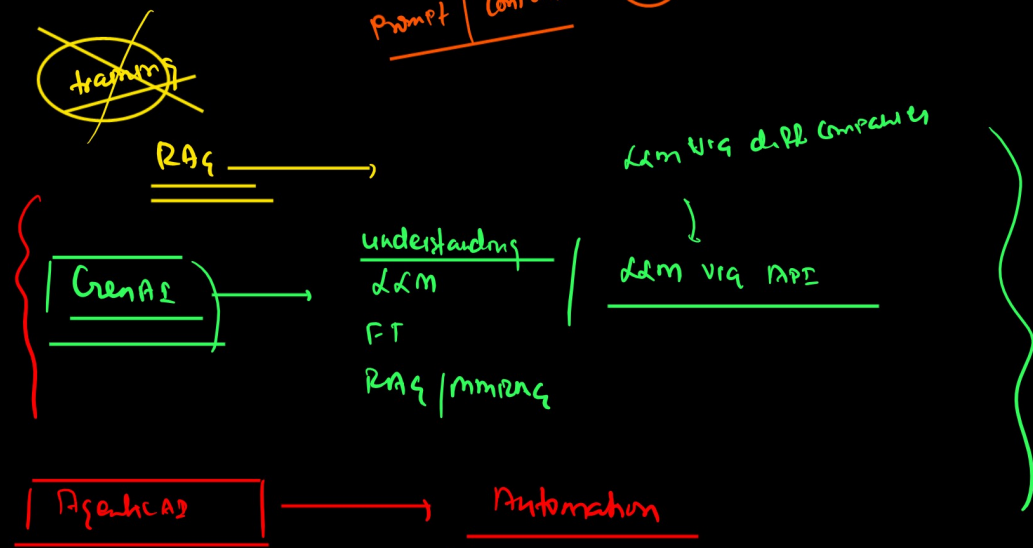
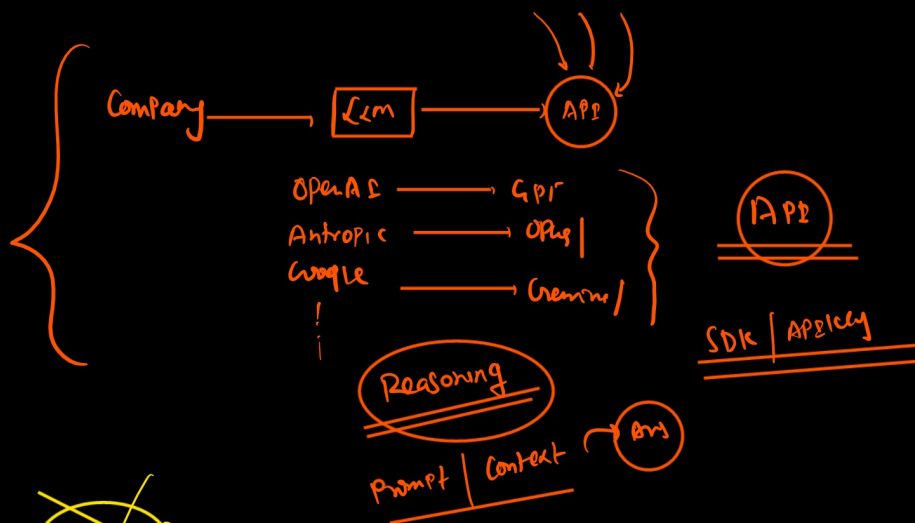
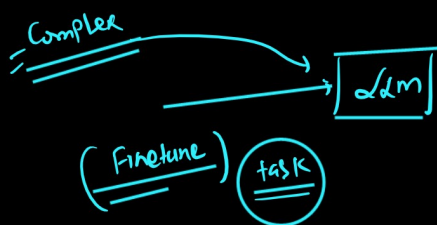
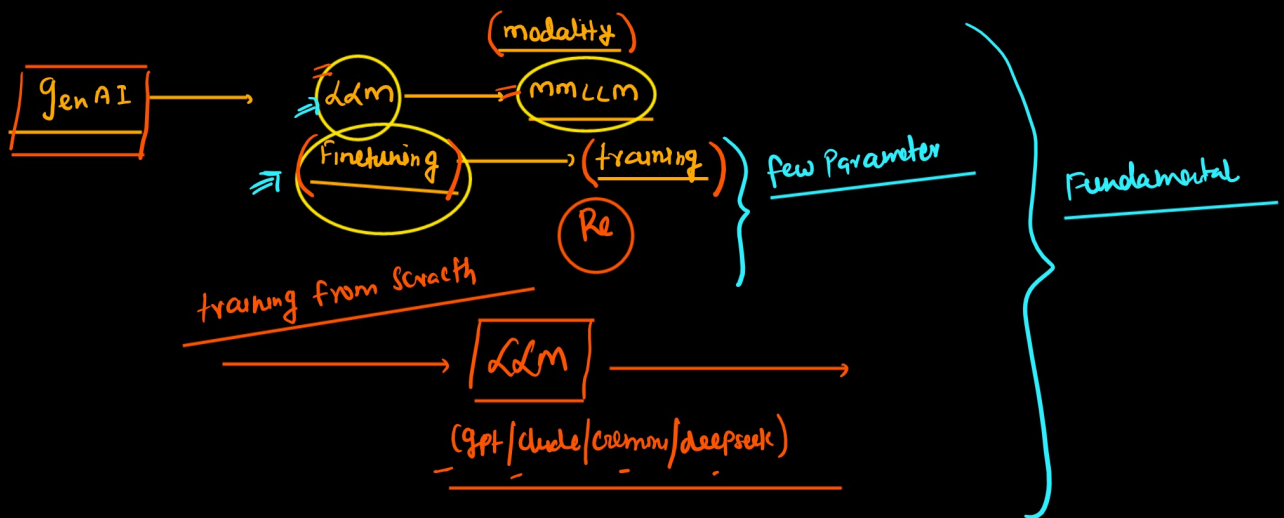
Langgraph

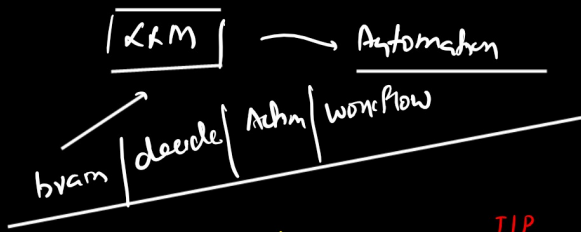
Don't create it using Langgraph

I have a inbuilt class for you → Langchain dev

Create - Agent C)

React Agent

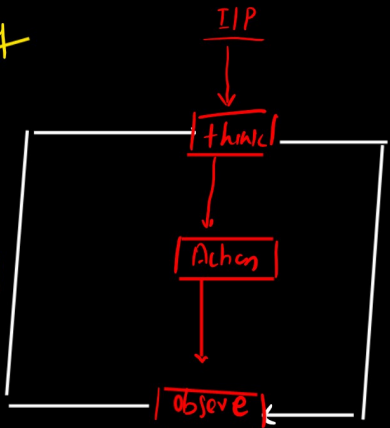




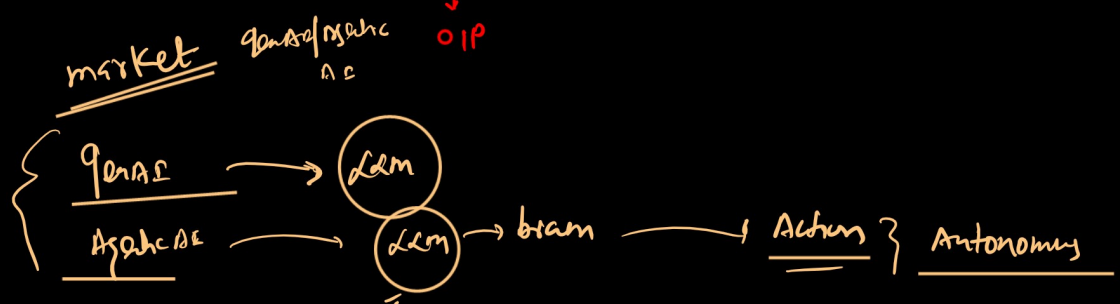
chat-gpt-
 Copilot → GitHub
 CodeR
 Claude code

Langchain inc → Company

Langgraph
 orchestration

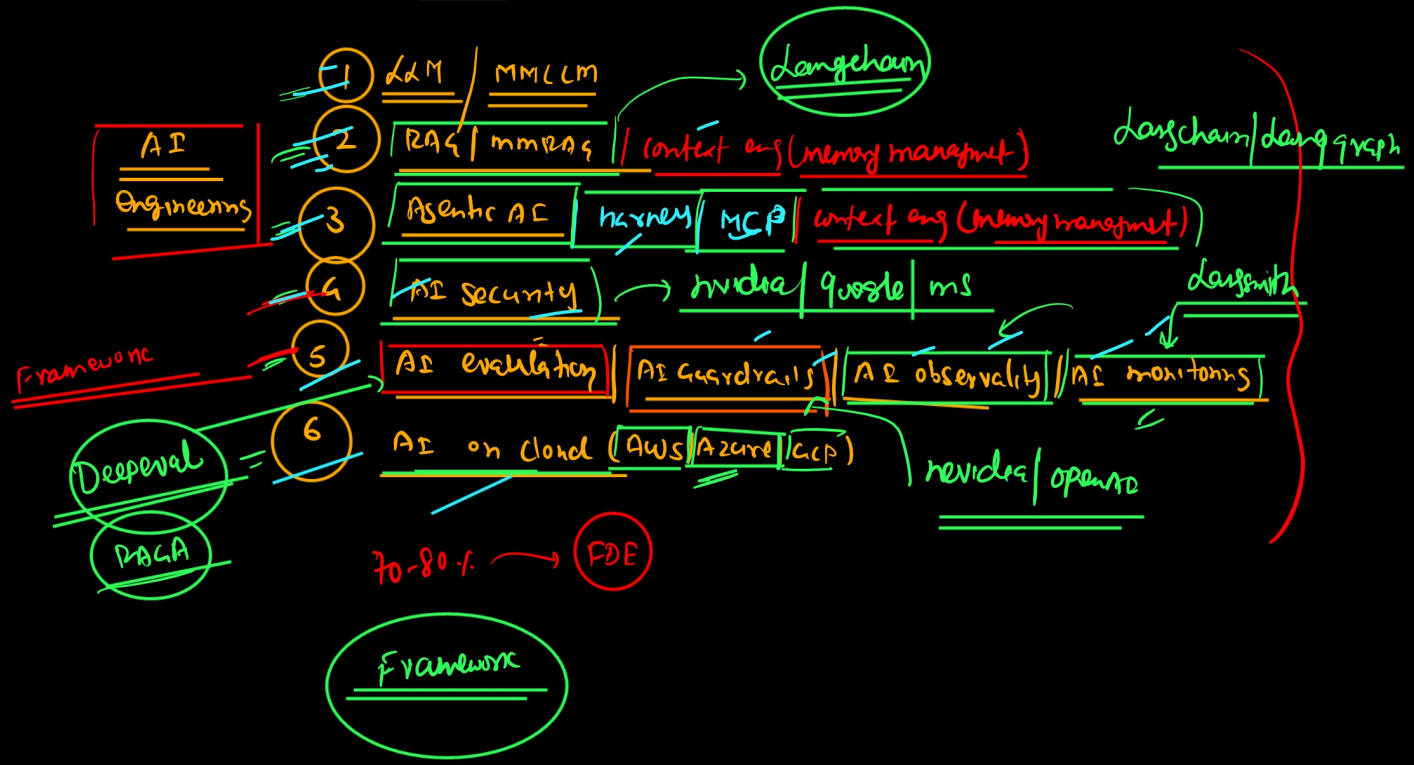


{ deterministic }
 { Probabilistic } → Asynchronous



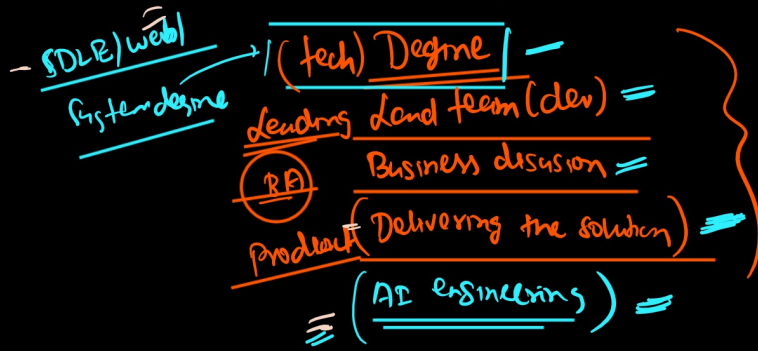
FDE

0-25 of



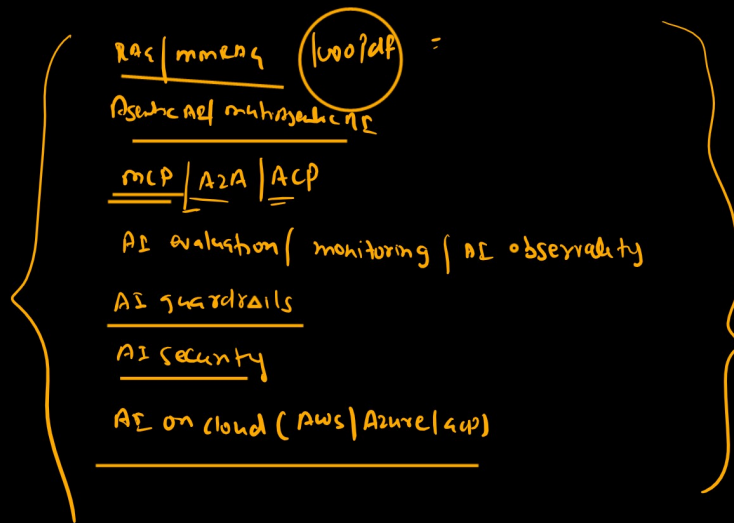
FDE tech lead | tech architect | tech consultant

Level





AI en



Scalable

messy

Agent/Agentic/Multiautentic AI

Autonomous system

Langchain Inc.

Agents

Langchain

create-agent()

middleware

tool system

Simple Agent

inbuilt

DeepAgent

Complex Agentic System

long horizon task

long running task

Complex agent

Langgraph

Complex agent / Simple agent / deterministic workflow

from scratch

Customizable over here

Deep_Agent.py

Agent

understand task

create plan

select tool

Delegate task

manage execution

Research Agent

Subagent

Analyst Agent

Subagent

This Agents can call tool

Custom tool

tool.py

deep agent → openSource

Langchain developer / community

inbuilt class

create-deep-agent()

Several Parameter

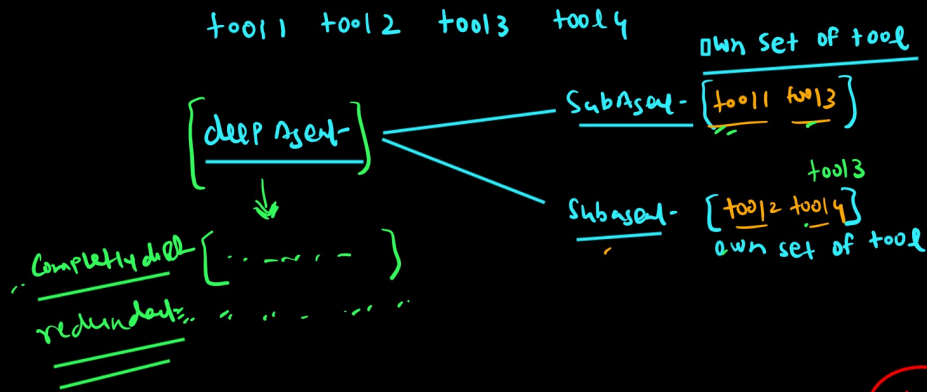
Human-approval

Valid FINAL result (Pydantic)

TOOLS $\longrightarrow$ extranal Functionality (Any)

we can expose as a tool ?

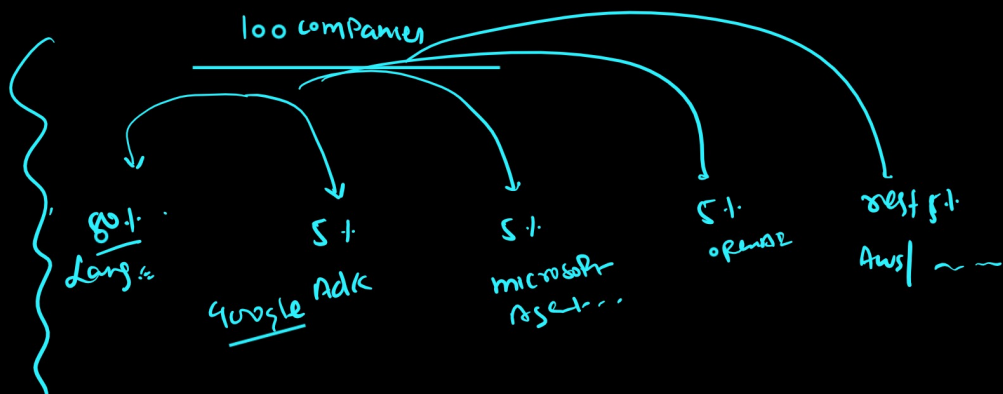
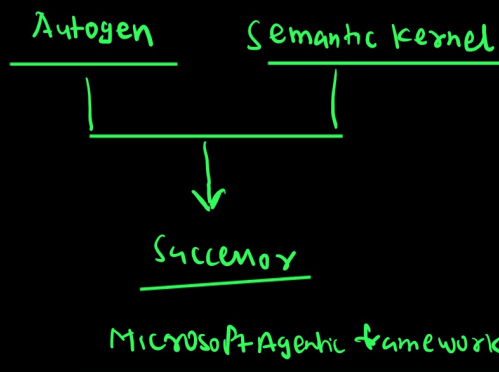
a ^{tool} Function

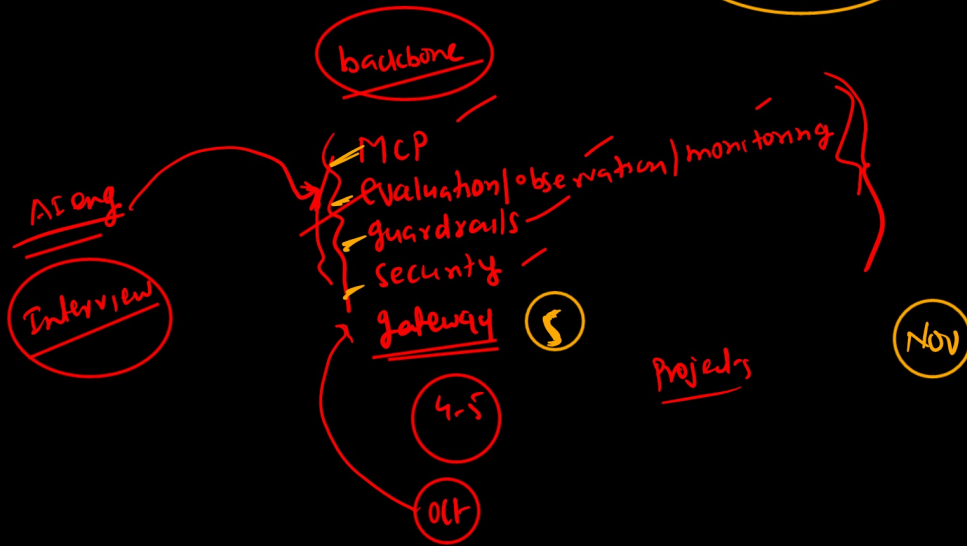
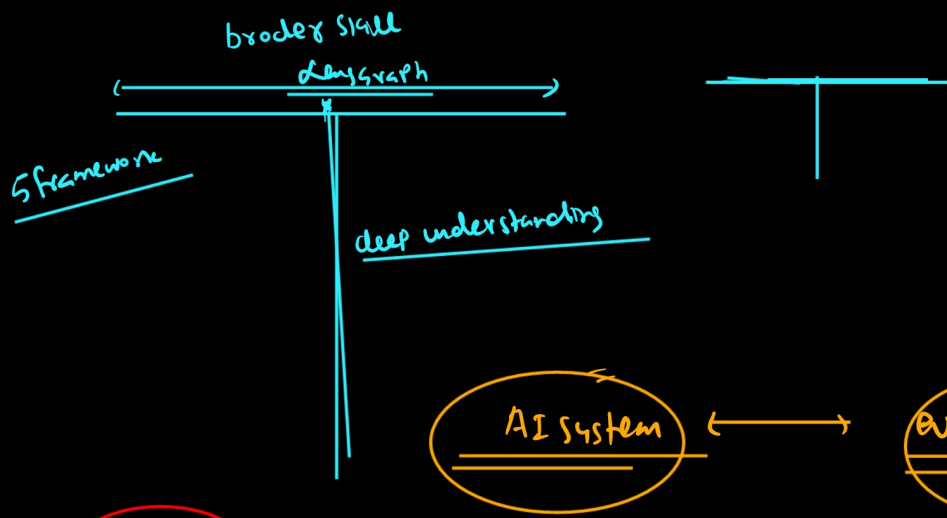


SKILL $\longrightarrow$ Set of Instruction

which generally we write inside .md

CLAUDE SKILL $\longrightarrow$ instruction (.md)





Langchain

Runnable

LCEL

Langgraph orchestration

Sequence

Parallel

lambda

Branch

Assign

managing history

Pass through